Continuous Integration (CI)

Lecture 3

greenwich.edu.vn

UNIVERSITY OF GREENWICH

Alliance with FPT Education

Lesson's Agenda

Section 1: CI Concepts

Section 2: GitHub Actions Anatomy

Section 3: Building the pipeline

Section 4: Wrap Up

Section 1: CI Concepts

Life Before CI

The Manual Pain

Developer finishes code ⇨ Manually runs tests
⇨ Manually builds artifacts ⇨ Manually uploads.

Risks:

"I forgot to run tests."

"It works on my machine but fails on the server."

Wasted time waiting for builds.

What is CI ?

The practice of merging code changes frequently (daily) into a shared repository.

Every merge triggers an **Automated Sequence**:

Build the code

Run tests

Report status

The CI “Gatekeeper”

Protecting the Codebase

CI acts as a security guard for the main branch.

Red Build ❌: Tests failed. Merge is blocked.

Green Build ✅: Safe to merge.

Goal: "Fail Fast". Find bugs immediately after committing.

Why GitHub Actions?

Tool Choice

Integrated: Built right into GitHub. No external servers (like Jenkins).

Free: Generous free tier for students and public repos.

Community: Thousands of pre-built "**Actions**" in the Marketplace.

Configuration: Defined as Code (YAML).

UNIVERSITY OF GREENWICH

Alliance with FPT Education

Section 2: GitHub Actions Anatomy

Core Components

How it works ?

Workflow: The automated process (**.yml** file).

Event: What triggers it? (Push, PR).

Job: A set of steps running on a machine.

Step: Individual task within a job.

The Workflow File

YAML file

Stored in **.github/workflows/** directory.

Example: **.github/workflows/ci.yml**.

Must be in the root of your repository.

Triggers (Events)

When to run ?

push: Runs when code is pushed.

pull_request: Runs when a PR is opened/updated
(Critical for Code Review).

workflow_dispatch: Adds a button to run manually.

```yaml
on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
```

Runners

The Execution Environment

A **Runner** is a virtual machine hosted by GitHub.

It is a fresh, clean computer created just for your job.

It is destroyed immediately after the job finishes.

Options: ubuntu-latest, windows-latest, macos-latest.

```
runs-on: ubuntu-latest
```

Jobs

Grouping Tasks

A workflow can have multiple jobs (e.g., *test, build, deploy*).

Parallelism: Jobs run in parallel by default.

Dependency: Use needs: test to make deploy wait for test.

Steps

The Task List

Inside a job, steps run sequentially.

Type 1 - Command: run: npm install
(Executes shell command).

Type 2 - Action: uses: actions/checkout@v3
(Uses a pre-made script).

GitHub Actions Marketplace

Don’t reinvent the Wheel

The Marketplace has actions for everything.

Need Node? **actions/setup-node**.

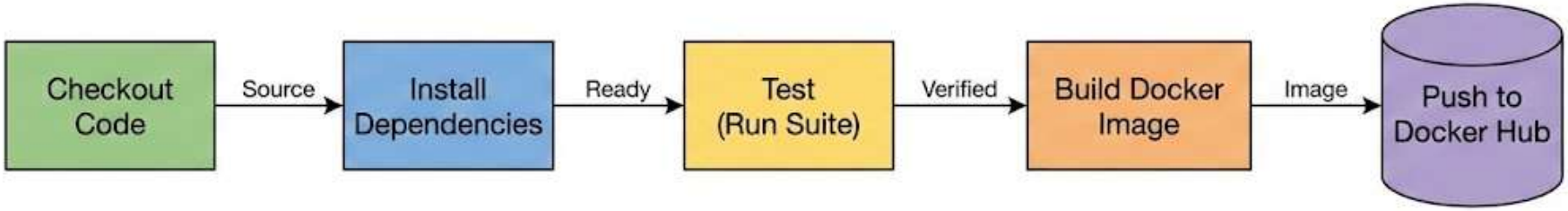
Need Docker? **docker/login-action**.

Need AWS? **aws-actions/configure-aws-credentials**.

Section 3: Building the pipeline

Pipeline Overview

- This is the standard pipeline for modern web apps.
  1. **Checkout:** Downloads the code to the runner.
  2. **Setup Environment:** Prepares the runtime environment (e.g., Node.js).
  3. **Install Dependencies:** Installs the necessary packages using `npm ci`.
  4. **Testing:** Runs the test suite to ensure the code is correct.
  5. **Docker Login:** Logs into Docker to push images.
  6. **Build & Push:** Builds the Docker image and uploads it to the registry.



Step 1 – Checkout

- The Runner is empty.
- This step downloads your Git repository code onto the Runner.
- Without this, you have no code to test!

```yaml
steps:
  - uses: actions/checkout@v3
```

Step 2 – Setup Environment

Setting up Node.js

- Prepares the language runtime.

```yaml
- uses: actions/setup-node@v3
  with:
    node-version: 18
```

Step 3 – Install Dependencies

- **npm ci** (*Clean Install*) is faster and strictly follows **package-lock.json**
- Use this for CI servers instead of npm install.

```
run: npm ci
```

Step 4 – Testing

- Executes your test suite (Jest, Mocha, etc.).
- **The Rule:** If this command returns an error (exit code 1), the **entire workflow stops**.
- This prevents bad code from reaching the build stage.

```
run: npm test
```

Putting It Together: ci.yml

Example CI workflow: build-and-test

- CI runs on every Pull Request to **main**
- **services.db** runs PostgreSQL in the CI runner (no external DB needed)
- Tests connect to **localhost:5432** using environment variables

```yaml
name: CI - Build & Test

on:
  pull_request:
    branches: [ main ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest

    services:
      db:
        image: postgres:15-alpine
        env:
          POSTGRES_USER: postgres
          POSTGRES_PASSWORD: postgres_password
          POSTGRES_DB: tododb
        ports:
          - 5432:5432
        options: >-
          --health-cmd="pg_isready -U postgres"
          --health-interval=10s
          --health-timeout=5s
          --health-retries=5
```

```yaml
steps:
  - name: Checkout code
    uses: actions/checkout@v4

  - name: Setup Node.js
    uses: actions/setup-node@v4
    with:
      node-version: 18

  - name: Install dependencies
    run: npm ci

  - name: Run tests
    env:
      DB_HOST: localhost
      DB_PORT: 5432
      DB_USER: postgres
      DB_PASSWORD: postgres_password
    run: npm test
```

Step 5 – Docker Login

To push an image, we must log in to Docker Hub.

Security Risk: Do NOT put passwords in the YAML file.

Solution: Use GitHub Secrets.

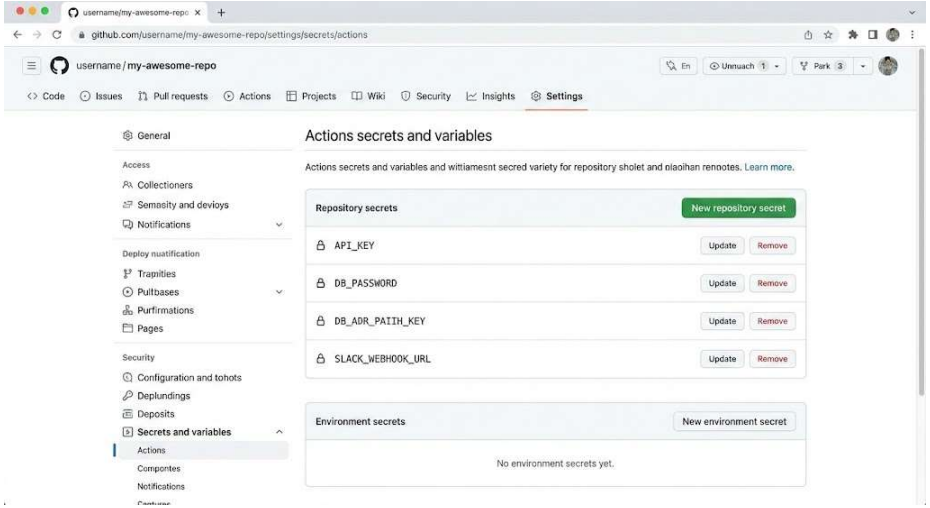
Managing Secrets

GitHub Secrets

Create DOCKER_USERNAME and DOCKER_PASSWORD

Access in YAML: ${{ secrets.DOCKER_PASSWORD }}

GitHub hides these values in logs (***).



Step 6 – Build & Push

- Builds the **Dockerfile**.
- Uploads the image to the registry.

```yaml
- uses: docker/build-push-action@v4
  with:
    push: true
    tags: myuser/myapp:latest
```

Step 5-6: Build & Push Docker Image

Example: build and push Docker image

- Docker Hub credentials must be stored in GitHub Secrets (never in YAML).
- Image name should match the group's repo/image naming convention

```yaml
- name: Log in to Docker Hub
  uses: docker/login-action@v3
  with:
    username: ${{ secrets.DOCKER_HUB_USERNAME }}
    password: ${{ secrets.DOCKER_HUB_PASSWORD }}

- name: Build and push image
  uses: docker/build-push-action@v6
  with:
    push: true
    tags: ${{ secrets.DOCKER_HUB_USERNAME }}/devops-todo:latest
```

Caching (Optimization)

Speeding it up

Downloading **node_modules** every time is slow.

Cache Action: actions/cache can save dependencies between runs.

Faster feedback = Happier developers.

Caching Node Modules in CI

NodeJS dependency caching

- Cache key is based on **package-lock.json**, so cache is invalidated when deps change
- This speed up **npm ci** in repeated runs

```yaml
- name: Cache node_modules
  uses: actions/cache@v4
  with:
    path: |
      ~/.npm
      node_modules
    key: ${{ runner.os }}-node-${{ hashFiles('package-lock.json') }}
    restore-keys: |
      ${{ runner.os }}-node-
```

Artifacts

Sometimes you want to save the output
(e.g., a .jar file or a compiled website).

Use **actions/upload-artifact** to store files for download later.

Section 4: Wrap Up

Branch Protection Rules

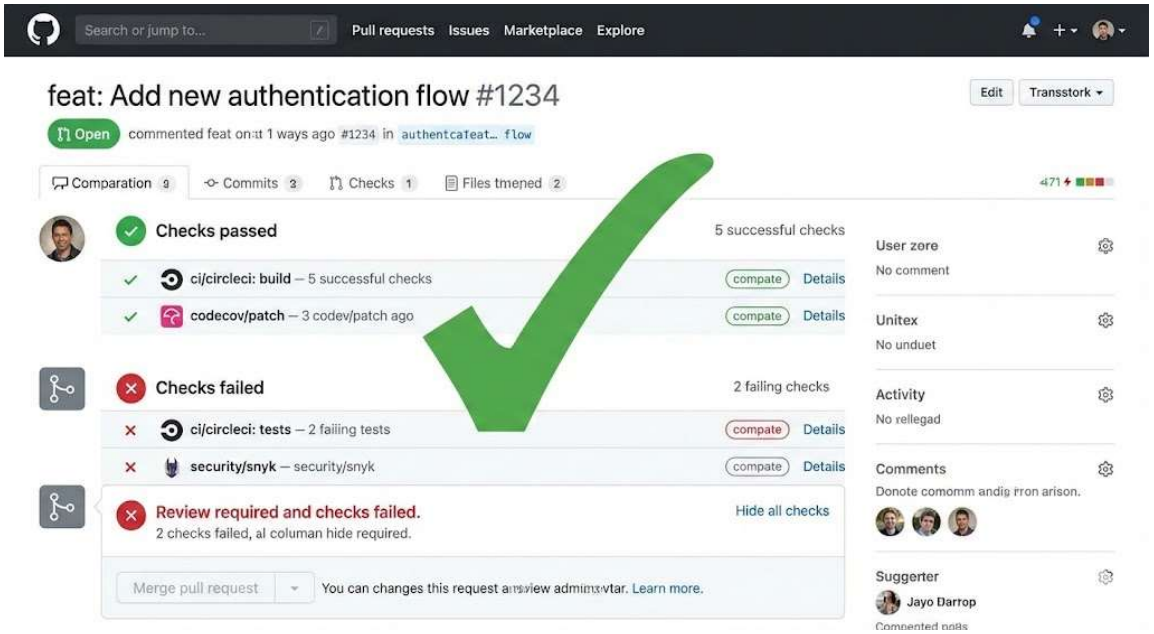
Enforcing CI

1. Go to GitHub Repo Settings ⇨ Branches.
2. Enable "*Require status checks to pass before merging*".
3. Select your *build-and-test* job.
4. Now, nobody can merge broken code!

Status Indicators

Visibility

- This provides immediate feedback to the developer.



Debugging Failures

- Click on the "**Details**" button in the PR.
- Read the console logs.
- Common errors:
  - Syntax error in code.
  - Test logic failure.
  - Missing Secret.
  - YAML indentation error.

UNIVERSITY OF GREENWICH
Alliance with FPT Education

Local Testing

Test Locally First

Don't treat the CI server as your testing ground.

Run npm test and docker build on your laptop *before* pushing.

"Respect the CI resources."

Adding Deploy Job (CD Preview)

Deploy job in CI

- **needs: build-and-test** ensures CD only runs if CI passed.
- **if: github.ref == 'refs/heads/main'** prevents deployments from PR branches.

```yaml
deploy:
  needs: build-and-test
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest

  steps:
    - name: Deploy via SSH
      uses: appleboy/ssh-action@v1
      with:
        host: ${{ secrets.HOST }}
        username: ${{ secrets.USERNAME }}
        key: ${{ secrets.SSH_PRIVATE_KEY }}
        script: |
          cd /path/to/your-project
          git pull origin main
          docker-compose pull
          docker-compose up -d
```

Broken Test Example

Fail ⇨ Fix ⇨ Pass

- First, intentionally make the test *wrong* so CI fails
- Then fix the application and/or test so the pipeline turns *green*

```
// tests/todos.test.js
it('POST /api/todos should fail without title', async () => {
  const res = await request(app)
    .post('/api/todos')
    .send({})
    .expect(400); // EXPECT 400 (Bad Request), not 500

  expect(res.body.error).toBe('Title is required');
});
```

Key Takeaways

- CI automates integration.
- GitHub Actions uses YAML workflows.
- Secrets keep passwords safe.
- Status Checks protect the main branch.

UNIVERSITY OF GREENWICH

Alliance with FPT Education